

Document de veille technologique, d'audit et de recommandations

Le plan présenté dans ce document est proposé à titre d'exemple, illustrant la structure attendue dans le cadre d'un projet réel.

Dans le contexte de cet exercice, ne cherchez pas à détailler chaque section de manière excessive, cela prendrait trop de temps.

Considérez ce document comme un guide plutôt qu'une liste d'attentes strictes à respecter à la lettre.

1. Introduction.....	2
1.1 Objectif.....	2
1.2 Rappel du contexte.....	2
2. Rapport de veille technologique.....	2
2.1 Méthodologie.....	2
2.2 Technologies évaluées.....	2
2.2.1 <Technologie X>.....	2
2.3 Matrices comparatives.....	2
2.3.1 <Catégorie X>.....	2
3. Audit des processus de développement.....	3
3.1 Objectifs.....	3
3.2 Méthodologie.....	3
3.3. Processus audités.....	4
3.3.1 Gestion du code et intégration continue.....	4
3.3.2 Dépendances, tests et builds.....	4
3.3.3 Conteneurisation et transmission aux opérations.....	4
3.3.4 Qualité, sécurité et traçabilité.....	5
3.3.5 Configuration d'exécution et données.....	5
3.4 Synthèse.....	6
4. Recommandations.....	6
4.1 Objectifs.....	6
4.2 Recommandations pour l'automatisation et la chaîne CI.....	6
4.2.1 <Recommandation 1>.....	6
5. Conclusion.....	7
5.1 Synthèse des constats.....	7
5.2 Prochaines étapes.....	7

1. Introduction

1.1 Objectif

Décrire l'objectif général du document et son importance pour le projet de l'entreprise.

1.2 Rappel du contexte

Fournir le contexte dans lequel cette veille technologique est réalisée, notamment les motivations à l'origine de celle-ci et les besoins spécifiques au contexte du projet.

2. Rapport de veille technologique

2.1 Méthodologie

Décrire les méthodes (essayer de modéliser l'approche qui a été suivie pour la recherche d'information) et les sources utilisées pour la veille technologique (ex: bibliographie des ressources documentaires, numériques ou physiques)

2.2 Technologies évaluées

Lister les différentes technologies évaluées dans le cadre de la veille technologique.

2.2.1 <Technologie X>

- **Description** : Rédiger une courte description de la technologie.
- **Catégorisation** : Identifier et assigner une catégorie à la technologie dans l'optique de réaliser la matrice comparative.
- **Fonctionnalités clés** : Lister les fonctionnalités clés associées à la technologie.
- **Cas d'usages** : Décrire les cas d'usages nominaux associés à la technologie.
- **Références** : Lister les références documentaires (URL, identifiant ISBN) qui ont servi à la réalisation de la veille pour cette technologie.

2.3 Matrices comparatives

Créer des matrices comparatives par catégorie listant les avantages/inconvénients des différentes technologies évaluées.

2.3.1 <Catégorie X>

Comparé à	Technologie A	Technologie B
Technologie A		Avantages - Avantage B vs. A Inconvénients - Inconvénient B vs. A

Technologie B	Avantages - <i>Avantage A vs. B</i> Inconvénients - <i>Inconvénient A vs. B</i>	
---------------	--	--

3. Audit des processus de développement

3.1 Objectifs

L'audit évalue l'efficacité de la chaîne d'intégration continue et des pratiques de développement actuellement utilisées pour MicroCRM. Il cherche à :

- reconstituer le fonctionnement réel depuis la modification du code jusqu'à la transmission d'une version aux opérations ;
- distinguer les pratiques décrites par les équipes de celles qui sont effectivement configurées dans le dépôt ;
- identifier les opérations manuelles, répétitions, points de friction et goulets d'étranglement ;
- mesurer l'état initial des tests, de la couverture, des builds et de la conteneurisation ;
- repérer les lacunes de qualité, de sécurité, de traçabilité et de portabilité ;
- rapprocher les besoins communs ou spécifiques des équipes Dev et Ops avant toute veille ou recommandation.

Cette analyse établit une baseline vérifiable. Elle servira à comparer l'existant avec la future chaîne CI full-stack, sans présumer à ce stade des outils ou solutions qui seront retenus.

3.2 Méthodologie

L'audit a été réalisé le 26 juillet 2026 à partir de sources de natures différentes :

- exigences officielles : contexte et consignes OpenClassrooms ;
- informations déclaratives : sondages des équipes Dev et Ops ;
- documentation existante : README et notes fournies ;
- observations techniques : arborescence Git, pipeline .gitlab-ci.yml, configurations Angular, Spring Boot et Docker ;
- mesures reproductibles : installation des dépendances, tests, couverture, builds, construction des images et smoke tests contrôlés.

Les déclarations des équipes ont été utilisées pour comprendre les usages et difficultés, puis confrontées au dépôt lorsque cela était possible. Une affirmation technique est considérée comme confirmée lorsqu'elle renvoie à un fichier, une commande, un rapport ou un comportement reproduit. Les causes qui ne peuvent pas être prouvées restent présentées comme probables.

Les mesures ont été effectuées localement sur la branche dev au commit 526bd96cddd2903676988b56dfb2778667aa435, sans modifier le code. Elles constituent une baseline fonctionnelle et non une mesure de performance d'un runner GitLab ou d'un environnement de production. Aucun test de charge, déploiement AWS ou indicateur d'exploitation n'a été réalisé pendant cet audit.

3.3. Processus audités

3.3.1 Gestion du code et intégration continue

Description — MicroCRM est organisé en monorepo GitLab avec un frontend Angular et un backend Spring Boot. Le pipeline versionné comprend deux stages, test et build, et quatre jobs séparés pour les deux composants.

Forces — Le dépôt GitLab existe déjà ; les composants sont clairement séparés ; le frontend possède un fichier de verrouillage npm et le backend utilise le Gradle Wrapper. Les commandes principales sont présentes dans le README et dans la CI.

Faiblesses — Aucun workflow de branches, de merge request ou de validation avant intégration n'est documenté dans le dépôt. Le pipeline ne définit ni règles d'exécution selon les branches ou tags, ni cache, ni dépendances explicites entre jobs.

Risques — L'exécution peut être identique pour des événements qui devraient être traités différemment. L'absence de règles partagées et de traçabilité formalisée augmente le risque d'intégration incohérente.

Sources — README, .gitlab-ci.yml, état Git observé et sondage Dev.

3.3.2 Dépendances, tests et builds

Description — La CI installe les dépendances, lance les tests frontend et backend, puis reconstruit chaque composant dans des jobs distincts.

Forces — Les commandes actuelles réussissent : 8 tests frontend et 2 tests backend ont été reproduits sans échec. Les builds Angular et Gradle aboutissent. Une couverture frontend peut être générée localement.

Faiblesses — Les huit tests frontend vérifient surtout la création des composants et services ainsi que le titre de l'application. Le backend ne contient qu'un test de chargement du contexte et un test de repository. Aucun scénario HTTP ou CRUD complet n'est couvert. La couverture mesurée reste faible : 30,76 % des lignes et 9,52 % des branches. La CI ne publie ni rapport JUnit, ni couverture, ni artefact. L'installation npm est répétée et le build Gradle réexécute les deux tests déjà lancés par le job de test.

Risques — Des régressions fonctionnelles peuvent ne pas être détectées. Les répétitions allongent inutilement le pipeline, tandis que l'absence de rapports et d'artefacts empêche de conserver une preuve ou de réutiliser un build déjà validé.

Sources — Fichiers de tests, rapports Karma et Gradle, rapport de couverture, .gitlab-ci.yml et mesures D.4.

3.3.3 Conteneurisation et transmission aux opérations

Description — Le Dockerfile multi-stage peut produire trois cibles : frontend, backend et standalone. Selon les sondages, les Dev génèrent majoritairement les images manuellement, puis les Ops les contrôlent avec Trivy et les déploient manuellement sur un environnement de démonstration.

Forces — Docker constitue une interface commune aux deux équipes. Les trois cibles ont pu être construites localement. Le backend démarre et répond à l'API ; dans l'image standalone, Supervisor lance bien les processus frontend et backend.

Faiblesses — Le pipeline GitLab ne construit, ne publie et ne versionne aucune image.

Aucun digest ou manifeste ne relie une version au commit qui l'a produite. Les images s'exécutent avec l'utilisateur root par défaut et ne déclarent aucun healthcheck. Le backend écoute sur 8080 alors que son image expose 4200. Le frontend contient bien ses fichiers statiques, mais le Caddyfile actuel déclenche une redirection HTTPS qui n'a pas permis de valider son smoke test.

Risques — Les manipulations manuelles favorisent les erreurs et les écarts de version. Une image peut parvenir tardivement aux contrôles de sécurité ou être difficile à relier au code source. Les incohérences de ports et de routage peuvent rendre le service indisponible.

Sources — Dockerfile, Caddyfile, supervisor.ini, .gitlab-ci.yml, sondages Dev/Ops et smoke tests D.4.

3.3.4 Qualité, sécurité et traçabilité

Description — Le dépôt actuel exécute uniquement les tests et builds applicatifs. Les Ops déclarent utiliser Trivy manuellement avant le déploiement.

Forces — Les équipes ont déjà identifié l'intérêt d'un contrôle des images. Les dépendances npm signalent les vulnérabilités pendant l'installation, ce qui rend une première observation possible.

Faiblesses — Aucun contrôle SonarQube, analyse de dépendances structurée, détection de secrets ou scan d'image n'est intégré au pipeline. L'installation npm observée pendant le build Docker signale 64 vulnérabilités, dont une critique et 46 élevées. Le détail des avis n'a pas été collecté pendant l'audit. Les sorties de qualité et de sécurité ne sont pas conservées dans GitLab.

Risques — Une anomalie peut être découverte seulement après la transmission de l'image aux Ops, comme l'illustre le retard déjà déclaré dans les sondages. L'absence de rapport lié au commit complique le suivi de la dette et la justification d'une acceptation ou d'un refus.

Sources — .gitlab-ci.yml, sortie npm ci du build Docker, sondages Dev/Ops et configurations du dépôt.

3.3.5 Configuration d'exécution et données

Description — Le frontend appelle une URL d'API définie dans le code, tandis que le backend utilise HSQLDB sans configuration de persistance externe. Le backend applique une configuration CORS permissive.

Forces — L'API démarre et répond correctement sur le port 8080. Les prérequêtes CORS testées pour GET et PUT sont acceptées. L'environnement embarqué permet une démonstration locale simple.

Faiblesses — L'adresse `http://localhost:8080` est intégrée jusque dans le bundle frontend et ne s'adapte pas à un autre environnement. La base HSQLDB perd la donnée ajoutée lorsque le conteneur est remplacé. Toutes les origines CORS sont autorisées et le comportement runtime autorise davantage de méthodes que la liste personnalisée visible dans le code.

Risques — Le frontend peut contacter la mauvaise machine après déploiement. Les données peuvent être perdues lors d'un remplacement de conteneur. Une configuration CORS trop large augmente inutilement la surface d'exposition.

Sources — `config.ts`, bundle Angular, `application.properties`, configuration Spring Data REST et tests contrôlés D.4.

3.4 Synthèse

L'existant fournit une base exploitable : le code est centralisé dans GitLab, le frontend et le backend sont séparés, les dépendances sont reproductibles, les tests et builds actuels réussissent, et les trois cibles Docker peuvent être construites.

Cette base ne constitue toutefois pas encore une chaîne de livraison complète. Le pipeline s'arrête aux tests et aux builds, répète certaines opérations et ne conserve ni rapports, ni artefacts, ni images. La profondeur fonctionnelle des tests et la couverture sont faibles. Les contrôles de qualité et de sécurité interviennent tardivement ou ne sont pas automatisés. Enfin, plusieurs écarts d'exécution ont été reproduits : port Docker incohérent, configuration frontend liée à localhost, frontend non validé au smoke test et données HSQldb non persistantes.

Les sondages et l'observation du dépôt convergent donc sur quatre besoins : réduire les opérations manuelles, détecter plus tôt les problèmes de qualité et de sécurité, transmettre une image identifiable et vérifiée, et rendre la configuration portable entre les environnements. Ces besoins orienteront la veille technologique de l'étape suivante. Aucune solution n'est retenue par le présent audit.

Limites — Les durées ont été mesurées sur le poste local et non sur un runner GitLab. Aucun pipeline distant, test de charge, environnement AWS ou indicateur de production n'a été observé. Le détail des avis de sécurité npm reste à compléter dans un contexte réseau autorisé.

4. Recommandations

4.1 Objectifs

Décrire les objectifs liés aux recommandations formulées dans le rapport et leur importance pour l'amélioration de la chaîne CI.

4.2 Recommandations pour l'automatisation et la chaîne CI

Fournir la liste des recommandations pour la chaîne CI et les autres processus de développement qui pourraient profiter d'une automatisation, toutes basées sur les constats de l'audit.

4.2.1 <Recommandation 1>

- **Description** : Décrire en détail la recommandation.
- **Raisonnement** : Justifier le raisonnement à l'origine de cette recommandation, expliquant pourquoi celle-ci est importante.
- **Implémentation** : Décrire les étapes/actions nécessaires pour mettre en place cette recommandation.
- **Impacts** : Lister les bénéfices attendus suite à la mise en œuvre de cette recommandation.

5. Conclusion

5.1 Synthèse des constats

Récapituler les principaux constats de la veille technologique et de l'audit, ainsi que les recommandations proposées.

5.2 Prochaines étapes

Identifier les prochaines étapes pour la mise en œuvre des recommandations et proposer une trame de planification de celles-ci. Notez qu'il n'est pas nécessaire d'implémenter toutes ces recommandations dans le cadre de ce projet.